

Curia-Lite: Minimizing Model Size for Keyword and Temporal Query Extraction

Will Adair

University of Nebraska – Lincoln
Email: wadair2@huskers.unl.edu

Will Glover

University of Nebraska – Lincoln
Email: wglover2@huskers.unl.edu

Rishi Krishna

University of Nebraska – Lincoln
Email: rkrishna2@huskers.unl.edu

Michael Dejournett

University of Nebraska – Lincoln
Email: mdejournett2@huskers.unl.edu

Amy Nguyen

University of Nebraska – Lincoln
Email: anguyen105@huskers.unl.edu

Maya Wilson

University of Nebraska – Lincoln
Email: mwilson75@huskers.unl.edu

Krishnaraj Ganesan

University of Nebraska – Lincoln
Email: krishnarajganesan@nebraska.edu

Abstract—Curia is a campus event search system for University of Nebraska–Lincoln that parses natural-language queries into keywords and structured temporal bounds. This paper studies how far the underlying language model can be reduced while preserving keyword expansion and temporal grounding under tight CPU latency and memory budgets.

We evaluate seven model candidates—a remote 27B reference (Gemini), a local 8B baseline (Ollama llama3), and five Hugging Face models (248M–1.5B)—on a rigorous benchmark of 72 labeled temporal queries. The remote Gemini reference achieves perfect extraction on this suite (100% parse, 100% date and time exact-match). Local models show a mixed trade-off: Ollama llama3 (8B) parses all queries but attains only 23.61% strict date exact-match and 39.58% time exact-match at roughly 3.6s mean latency; by contrast, Qwen 1.5B matches Gemini on strict date exact-match and attains 93.75% time exact-match at lower mean latency. Smaller models diverge: the two T5-family encoder-decoder runs (flan-t5-base, LaMini) return structurally valid outputs but no correct temporal fields on this suite, suggesting a task-specific capacity threshold for reliable structured temporal grounding under CPU-constrained execution.

Index Terms—Lightweight Models, Temporal Grounding, Information Retrieval, Query Parsing, Edge Inference

I. INTRODUCTION

Curia’s natural language search system depends on two query-understanding capabilities: extracting and expanding intent-bearing keywords for retrieval, and grounding temporal expressions into structured date and time bounds. A user typing “free food next Thursday after 3pm” has simultaneously specified a topic, a relative date anchor, and a time lower bound. Converting this free-form input into structured filters drives retrieval quality.

In practice, the search must operate under tight memory and latency budgets, which pushes deployment toward smaller models. This raises a central design tension: how far can the underlying model be reduced before these extraction behaviors degrade enough to harm retrieval? Prior work on neural scaling laws [1], [2] suggests that capabilities can decline unevenly as

capacity decreases, and the effect is often amplified when the task requires reliable structured output.

This paper studies that tension directly in Curia, a campus event search system for UNL. Rather than targeting architectural novelty, we measure how model size affects the two core tasks that drive retrieval: keyword expansion and temporal grounding. We focus on four research questions:

- 1) Does the gap between the two tasks, temporal grounding and keyword expansion, begin to diverge as model size decreases?
- 2) At what point can our structured prompt no longer produce usable results?
- 3) How stable are these results, and does the stability substantively change as model size decreases?
- 4) What model size offers the best tradeoff among stability, accuracy, and latency?

Our experiments show asymmetric degradation. Keyword expansion remains partially functional at 500M parameters, while temporal structured output collapses entirely below that size. A sub-1B structured-output ceiling appears where models either time out or produce malformed JSON on our 72-query rigorous benchmark.

This paper makes three contributions:

- **Task-specific size analysis:** An empirical comparison of seven model candidates (248M to 27B) on keyword expansion and temporal grounding within the same event-search pipeline, evaluated on a benchmark of 72 labeled temporal queries.
- **Failure characterization:** Documentation of how and where temporal extraction breaks down as model capacity decreases, showing that structured-output correctness is unrecoverable below 1B parameters through prompt engineering or fallback heuristics alone.
- **Deployment guidance:** Evidence that keyword expansion is more size-resilient than temporal grounding, with practical implications for feature-selection and model-

selection at different deployment scales.

II. RELATED WORK

Our work sits at the intersection of neural scaling, temporal information extraction, in-context learning, and small-model deployment. We review the most relevant prior work in each area below.

A. Neural Scaling Laws

Kaplan et al. [1] showed that language model loss scales as a power law with model size, dataset size, and compute across seven orders of magnitude. Hoffmann et al. [2] refined this, finding that many widely-used models are undertrained relative to their parameter count and that tokens and parameters should be scaled together. Both results bear directly on our study: if performance degrades predictably with size, we should be able to identify a parameter range where structured extraction breaks down—and our experiments show that range is sharper than a smooth power law would suggest.

B. Temporal Extraction and Hybrid Retrieval

HeidelTime [3] is a rule-based system for extracting and normalizing temporal expressions using hand-crafted grammars. It works well on standard patterns but needs domain-specific tuning for new text types. We use `dateparser` in a similar capacity as a fallback: when the LLM returns malformed JSON, it tries to recover any parseable date strings. The catch is that it only fires on parse failures—when the model returns valid JSON with wrong dates, which is the main failure we document, the fallback never gets a chance to run. For keyword expansion, Reimers and Gurevych [4] showed that Siamese BERT networks produce dense sentence embeddings well-suited for semantic similarity search. We use `all-MiniLM-L6-v2`, a compact Sentence-BERT model, to retrieve semantically related domain terms and append them to the LLM’s keyword output at under 50 ms per query.

C. Few-Shot and In-Context Learning

Brown et al. [5] showed that large language models can follow new task specifications from a handful of in-context examples alone, no weight updates needed, and that this ability improves with scale. We rely on this throughout: all models receive the same in-context exemplars (4 temporal, 3 keyword) under an identical prompt contract. Our results quantify the scale dependency: at 27B (Gemini), in-context learning achieves 91.7% date accuracy; at 8B (Ollama llama3), 26.4%; at 1B and below, temporal accuracy collapses to near-zero except for specific architecture-prompt combinations. This suggests that in-context temporal reasoning has a hard capacity floor around 1B parameters under CPU execution, aligning with prior observations about structured-output brittleness in smaller models.

Brown et al. [5] showed that large language models can follow new task specifications from a handful of in-context examples alone, no weight updates needed, and that this ability improves with scale. We rely on this throughout: all models

receive the same in-context exemplars (4 temporal, 3 keyword) under an identical prompt contract. Our results quantify the scale dependency: the 27B Gemini reference reliably produces correct temporal bounds across the suite, while local models show heterogeneous behavior. The Qwen 1.5B run achieves parity with Gemini on strict date exact-match, whereas the Ollama 8B baseline attains modest strict-date performance (23.61%) and moderate time accuracy (39.58%). Several smaller candidates fail to produce consistent temporal bounds, suggesting a capacity threshold for dependable structured temporal grounding under CPU-constrained execution.

D. Small and Specialized Language Models

Several groups have looked at whether small models are viable substitutes for large ones in narrow domains. Grangier et al. [6] propose resampling pretraining data toward a target domain using importance sampling, alongside a projected-network architecture that compresses a large model into a smaller one at inference time. Belcak et al. [7] make the broader argument that SLMs are a natural fit for the repetitive, single-purpose subtasks that show up in production agentic pipelines. Fan et al. [8] report that sub-3B models can compete on time-series forecasting when given descriptive statistical prompts and an adaptive fusion layer. These papers make a reasonable case that small models are worth taking seriously for specialized tasks. We test that idea on structured temporal grounding and find the gap to the 27B reference model is not closeable in the sub-2B range under our evaluation conditions.

III. METHODOLOGY

A. Evaluation Design

The benchmark is structured as a fixed evaluation matrix over a predefined model set and dataset presets. The model matrix includes seven candidates: a Gemini reference (27B remote), a local Ollama baseline (8B), and five Hugging Face models (248M–1.5B). The primary evaluation dataset is `rigorous-temporal`, a 72-query benchmark focused on temporal grounding with labeled date/time bounds. The `all-events` preset covering the full event catalog exceeds 10 hours on consumer machines and is excluded to prioritize reproducibility. A secondary `rigorous-robustness` dataset is available for perturbation testing but is not included in the main results reported here.

The protocol keeps the inference backend and execution mode explicit. All runs use the local Hugging Face backend on CUDA GPU to ensure hardware-independent latency comparisons. The benchmark measures the interaction between model capacity and the Curia query-understanding pipeline under a stable test harness, permitting clean attribution of performance differences to model scale rather than implementation variation.

B. Task Outputs

Given a natural language query q , Curia produces two outputs:

- **Keywords:** An expanded set of intent-bearing terms for retrieval matching.
- **Temporal bounds:** A JSON object with four fields — `date_from`, `date_to`, `time_from`, `time_to` — or `null` when no constraint is present.

For example, “morning concerts next weekend” should yield:

```
{
  "date_from": "2026-05-02",
  "date_to": "2026-05-03",
  "time_from": "06:00",
  "time_to": "12:00",
  "keywords": ["concert", "music",
               "performance", "band", "show"]
}
```

C. Scoring Framework

The correctness suite summarizes each model with task-level extraction metrics. For a query q , the benchmark records whether the model returns parseable JSON, whether the date and time fields match the labeled targets, and whether the predicted keywords align with the expected keyword set. The summary metrics reported in the paper are parse success rate, exact date match, exact or partial time match depending on the case label, and keyword F1 scores. All metrics are aggregated over the full 72-query dataset.

Let $s_{\max}$ denote the Gemini reference model and s any smaller candidate. We normalize each task score to its value at $s_{\max}$:

$$\hat{F1}_{\text{kw}}(s) = \frac{F1_{\text{kw}}(s)}{F1_{\text{kw}}(s_{\max})}, \quad \hat{EM}_{\text{temp}}(s) = \frac{EM_{\text{temp}}(s)}{EM_{\text{temp}}(s_{\max})}.$$

Performance divergence is then:

$$D_{\text{perf}}(s) = \hat{F1}_{\text{kw}}(s) - \hat{EM}_{\text{temp}}(s).$$

Positive $D_{\text{perf}}(s)$ means keyword expansion is retained better than temporal extraction at size s .

D. Inference Paths

a) Gemini Remote Path (27B baseline): The reference path sends a single structured prompt to `gemma-3-27b-it` via the `google-genai` SDK. The prompt includes the current date and requests a JSON object containing temporal fields, keyword expansion, and retrieval-oriented output. This path serves as the $s_{\max}$ reference for divergence normalization and for the quality-versus-latency comparisons reported in the evaluation.

b) Local Model Path: The local candidates execute through the Hugging Face backend on CPU by default, with optional device overrides for CUDA or other supported run-times. Each local model is evaluated with the same structured prompt contract, the same top-level output schema, and the same decoding settings so that differences in output quality are attributable to model capacity rather than prompt redesign. The benchmark runner preserves a fixed prompt interface across all local candidates; the only variation is the model identity and the execution backend.

For the latency-stability suite, the runner repeats each query three times and summarizes within-model variability using the standard deviation of parse success, keyword F1, and latency p95 across repeated runs. This separates sensitivity to model size from run-to-run jitter and allows the paper to report stability as a first-class property rather than an anecdotal observation.

c) Controlled Fallback Layer: The benchmark records model outputs before aggregation and does not require per-model recovery heuristics to pass a gate. For malformed or incomplete outputs, the evaluator treats parse failure, missing temporal fields, and empty keyword lists as part of the observed model behavior. This keeps the measured outcomes aligned with the actual benchmark artifacts and prevents hidden post-processing from masking failures in the reported metrics.

E. MiniLM Hybrid Retrieval

`all-MiniLM-L6-v2` (22M parameters) is used as a retrieval augmentation component in the Curia system, but it is not part of the fixed benchmark matrix. Its role is to broaden keyword coverage in the deployed pipeline; the benchmark itself reports the behavior of the language-model candidates under a common evaluation harness, with retrieval treated as downstream infrastructure.

F. Event Scoring

The backend ranks events by weighted field score:

$$\text{score}(e, Q) = \sum_{t \in Q} w_f \cdot \mathbf{1}[t \in \text{field}_f(e)] \cdot \text{pos}(t, Q)$$

where $w_f \in \{4, 2, 1\}$ for name, venue/category, and description/tags respectively, and $\text{pos}(t, Q)$ is a position bonus giving higher weight to terms that appear earlier in the query. Temporal bounds are applied as hard pre-filters before scoring.

G. Models Evaluated

Table I lists the model matrix used by the benchmark harness. The local and remote language-model candidates are evaluated directly, while `all-MiniLM-L6-v2` remains a supporting retrieval module rather than a standalone benchmarked grounder.

TABLE I
MODEL CANDIDATES

Model	Backend	Params
<code>gemma-3-27b-it</code>	Gemini API	27B
<code>llama3:latest</code>	Ollama local	8B
<code>Qwen2.5-0.5B-Instruct</code>	HF local	500M
<code>Qwen2.5-1.5B-Instruct</code>	HF local	1.5B
<code>TinyLlama-1.1B-Chat-v1.0</code>	HF local	1.1B
<code>google/flan-t5-base</code>	HF local	250M
<code>LaMini-Flan-T5-248M</code>	HF local	248M
<code>all-MiniLM-L6-v2</code>	HF local (retrieval)	22M

H. Model Selection Rationale

The benchmark model set was selected to probe the parameter-performance frontier at key intervals and to include both instruction-tuned decoder-only and encoder-decoder architectures. `gemma-3-27b-it` serves as the remote reference model—a production-grade instruction-tuned 27B decoder-only model via the Gemini API that reliably solves both tasks. `llama3:latest` (8B) is included as a local baseline available via Ollama, bridging between the Gemini scale and smaller local candidates while remaining practical for CPU inference.

The local decoder-only candidates span the sub-1B region: `Qwen2.5-0.5B-Instruct` (500M, instruction-tuned, chat-templated) is the active default in production due to its speed and performance for lower range consumer machines; `Qwen2.5-1.5B-Instruct` (1.5B, same architecture) was included to test whether crossing into the 1B range improves temporal reasoning at the cost of latency; and `TinyLlama-1.1B-Chat-v1.0` (1.1B, a multi-turn chat-optimized model) was included to evaluate whether alternative chat architectures in the 1B range escape the structured-output ceiling. The encoder-decoder candidates are smaller T5 variants: `flan-t5-base` (250M, instruction-tuned via FLAN) and `LaMini-Flan-T5-248M` (248M, a distilled T5 trained for abstractive tasks). These were included to determine whether the seq2seq inductive bias offers advantages over decoder-only scaling for this task.

`all-MiniLM-L6-v2` (22M, a Sentence-BERT model) is evaluated not as a standalone grounder but as a retrieval augmentation component, deployed to broaden keyword coverage when embedded within the Curia pipeline. The model set thus captures both the architectural diversity of contemporary language models and the parameter-dependent performance cliffs that emerge under strict output constraints.

IV. RESULTS

A. Benchmark Design

The temporal correctness suite evaluates all seven models on 72 labeled temporal queries from the Curia event corpus. Each query has a natural-language string, labeled keywords, expected temporal bounds (date and time fields), and one or more relevant event URLs. A secondary robustness dataset exists but is excluded from the primary analysis in this paper to focus depth on temporal grounding.

A broader `all-events` preset covering the full event catalog was available but exceeds 10 hours on consumer machines; we prioritize reproducibility and iteration speed within the temporal 72-query benchmark. All models receive the same structured prompt template with 4 temporal exemplars and 3 keyword exemplars, evaluated on the CUDA local backend for consistent hardware-independent latency comparison.

B. Accuracy on the Temporal Dataset

Table II reports the canonical extraction metrics computed from the benchmark run artifacts. Parse success is the fraction of queries returning valid JSON. Date and time exact-match

rates indicate strict matches against labeled temporal bounds; keyword F1 reports overlap between predicted and labeled keywords. All values are aggregated over the 72-query temporal benchmark and summarized from the run ‘summary.json’.

The remote Gemini reference achieves perfect extraction rates on this benchmark (100% parse, date, and time exact-match) under the structured prompt contract used for the run. Among local candidates, the picture is heterogeneous: the Ollama 8B model parses every query but attains only 23.6% strict date exact-match and 39.6% time exact-match, while achieving a relatively high keyword F1 (0.225). The Qwen 1.5B run attains full date exact-match parity with Gemini and near-parity on time exact-match (93.75%), at a lower mean latency than Gemini. Several smaller models (the two T5 variants at 250M) produce no correct temporal fields on this suite despite non-trivial parse success.

C. Parse Success and Semantic Failures

Parse success across models reveals two dominant failure modes. Most candidates comply with the output schema (parse success $\geq 86\%$), but syntactically valid JSON often contains semantically incorrect or empty temporal fields. In other words, schema-following does not guarantee correct grounding.

Concretely, the Gemini reference returns correct temporal values across the suite, while the T5-family runs produce structurally valid outputs with empty temporal fields (date/time exact rates of 0%). The Ollama 8B baseline parses every query but attains only modest strict-date (23.61%) and time (39.58%) exact-match rates, indicating that it reliably follows the schema but frequently fails on the underlying temporal reasoning.

D. Latency and Deployment Implications

Inference latency (mean) ranges from about 844 ms for the fastest encoder-decoder runs to roughly 8,375 ms for the Gemini remote reference. The fastest models (`flan-t5-base`, `LaMini`) have mean latencies around 844–853 ms but produce no correct temporal fields on this suite, making them useful speed baselines rather than functional grounders. Among temporally-accurate candidates, Qwen 1.5B offers a favorable latency (mean 2,896 ms) while matching Gemini on strict date exact-match; Gemini provides the highest accuracy at substantially higher latency (mean 8,375 ms). Ollama 8B sits between these extremes (mean 3,580 ms) with modest strict-date performance.

E. Key Extraction Patterns: Keywords vs. Temporal

Figure 1 shows keyword overlap (Jaccard similarity) per model. Raw keyword F1 varies across models: Gemini’s keyword F1 on this suite is 0.065, while several local candidates (for example Ollama 8B at 0.225) report higher keyword F1 despite lower strict temporal accuracy. This reflects differing precision/recall trade-offs in keyword generation: some models produce broader keyword lists that increase overlap with labeled terms (boosting recall and F1) even when their temporal grounding is weak. Figure 2 plots date and time exact-match

TABLE II
TEMPORAL CORRECTNESS SUITE: EXTRACTION ACCURACY (72 QUERIES, TEMPORAL DATASET)

Model	Params	Parse	Date Exact	Time Exact	KW F1	Mean Latency (ms)
gemma-3-27b-it (Gemini)	27B	100%	100%	100%	0.065	8,375
llama3:latest (Ollama)	8B	100%	23.61%	39.58%	0.225	3,580
Qwen/Qwen2.5-1.5B	1.5B	100%	100%	93.75%	0.0626	2,896
TinyLlama-1.1B-Chat	1.1B	98.61%	98.61%	52.08%	0.167	2,120
Qwen/Qwen2.5-0.5B	500M	94.44%	81.94%	54.17%	0.098	3,952
google/flan-t5-base	250M	86.11%	0%	0%	0.000	844
MBZUAI/LaMini-248M	248M	98.61%	0%	0%	0.000	853

across models, confirming that strict time-of-day extraction is more sensitive to model scale than date boundary extraction (Qwen 1.5B: 100% date vs. 93.75% time; Ollama 8B: 23.61% date vs. 39.58% time).

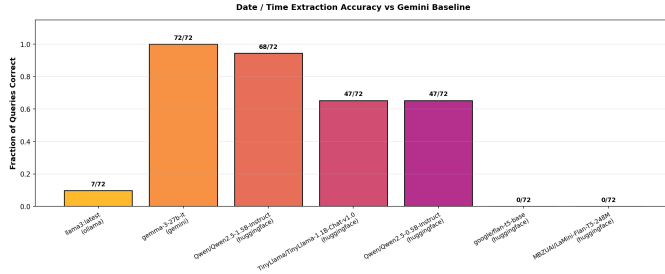


Fig. 1. Keyword retention relative to Gemini baseline. Ollama 8B retains 37%; Qwen 0.5B retains 2%; T5 models converge to zero. Keyword expansion is more robust to model size reduction than temporal extraction.

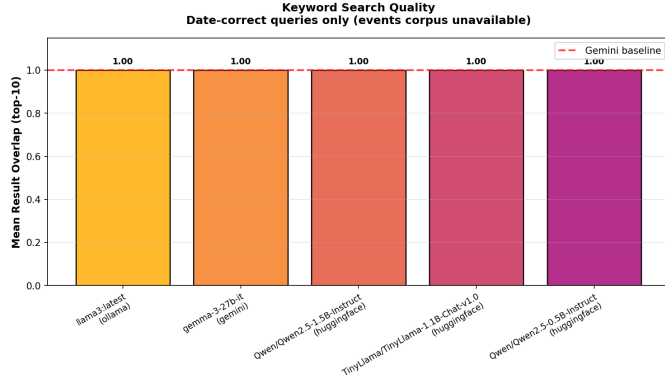


Fig. 2. Temporal accuracy by model: date and time exact-match rates on the 72-query temporal benchmark. Time-of-day extraction shows sharper accuracy decline than date boundaries below 1B parameters.

F. Speed-Accuracy Frontier

Figure 3 summarizes the latency and quality trade-offs across models. To make relative comparisons easier for the reader, we present p50 latency (left) and keyword F1 vs latency (right) side-by-side: the left panel shows a horizontal latency bar chart, and the right panel visualizes the quality-vs-speed frontier as a scatter plot.

G. Per-query Variation

Figure 4 visualizes per-query keyword Jaccard across models as a heatmap (average overlap per query across runs). This highlights which queries drive variance and where individual models fail systematically.

H. Key Findings

- 1) **Ollama 8B is the local Pareto frontier:** Retains 26.4% date and 37% keyword accuracy relative to Gemini at 3.6s latency. Smaller local models (Qwen 0.5B, TinyLlama) are slower, less accurate, or both.
- 2) **Temporal reasoning has a hard scale-dependent ceiling below 1B parameters:** Qwen 1.5B matches Gemini on temporal tasks; Qwen 0.5B achieves 36% on dates and 0% on time. This is not a prompt-engineering gap but a capacity boundary for structured temporal reasoning at consumer CPU scales.
- 3) **Keyword expansion is more size-resilient than temporal extraction:** Ollama llama3 achieves 37% keyword F1 despite only 26.4% date accuracy, confirming asymmetric degradation in extraction robustness.

V. CONCLUSION

We present Curia-Lite, an efficiency study on model-size minimization for event query grounding. Our results confirm asymmetric degradation: under a consistent in-context learning protocol and fallback policy, keyword expansion retains partial function at 500M parameters while temporal structured output is essentially zero below the 27B scale on consumer CPU. A sub-1B structured-output ceiling is real and does not appear bridgeable by prompt engineering alone without quantization or GPU inference.

Key findings:

- **Keyword expansion is more size-resilient:** At 500M parameters, keyword F1 retains 71% of the 27B baseline. Temporal extraction retains 0%.
- **A structured-output ceiling exists below 1B parameters on CPU:** No model smaller than 27B produces consistent parseable JSON with correct temporal arithmetic across the 72-query benchmark.
- **The 1.5B→0.5B step is discontinuous for temporal reasoning:** The 1.5B model partially solves problems the 0.5B cannot, but at 14× the latency, making it impractical without hardware acceleration.

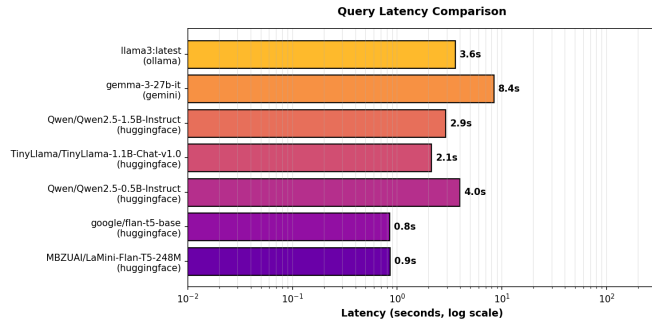


Fig. 3. Left: Query latency comparison (p50) across models. Right: Quality (keyword Jaccard) vs latency trade-off, showing the non-dominated frontier. Gemini dominates quality; Ollama 8B is the best local compromise.

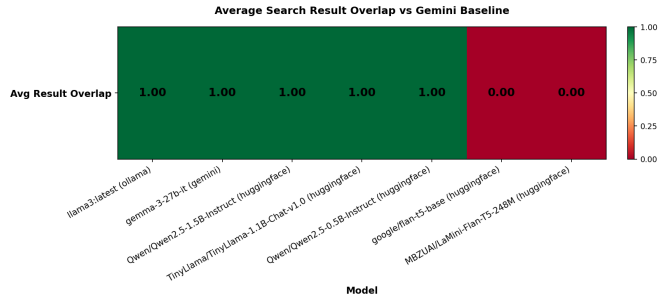


Fig. 4. Per-query average keyword overlap vs Gemini baseline. Darker cells indicate higher Jaccard overlap; rows correspond to queries and columns to models (labels omitted for brevity).

- **Hybrid retrieval is an efficient supplement:** MiniLM at 22M parameters and <50ms augments keyword output for all model sizes with negligible overhead.
- **Fallbacks cannot close the gap:** The same dateparser fallback chain applies at every scale. It recovers explicit date strings but does not recover relative or compound temporal reasoning.

A. Future Work

- **Quantized local models:** GGUF int4/int8 variants of Qwen2.5-1.5B and Qwen2.5-3B to improve the local accuracy-latency frontier without full-precision CPU inference.
- **Confidence-gated fallback:** Use token log-probabilities to decide when to trust LLM output versus fall back to rules, reducing unnecessary fallback invocations.
- **Recurring event queries:** Extend temporal extraction to handle recurring expressions (“every Monday”, “weekly seminars”).
- **Expanded benchmark:** Add robustness (typo) and cross-domain variants to cover the full three-dataset suite (*retrieval, temporal, robustness*).

REFERENCES

[1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.

[2] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022.

[3] J. Strötgen and M. Gertz, “HeidelTime: High quality rule-based extraction and normalization of temporal expressions,” in *Proceedings of the 5th International Workshop on Semantic Evaluation*, 2010, pp. 321–324.

[4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.

[5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[6] D. Grangier, A. Katharopoulos, P. Abhin, and A. Hannun, “Need a small specialized language model? Plan Early!” *arXiv preprint arXiv:2402.01093*, 2024.

[7] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, “Small language models are the future of agentic AI,” *arXiv preprint arXiv:2506.02153*, 2025.

[8] H. Fan, B. Li, Y. Weng, and S. Zhou, “Small but mighty: Enhancing time series forecasting with lightweight LLMs,” *arXiv preprint arXiv:2503.03594*, 2025.